

REQUEST FOR PROPOSAL

Escape the 90s

2D Side-Scrolling Game Development Project

Unity / C# | PC Build | Full-Project Procurement Scope

RFP No. E90S-2026-01

Issue Date: September 19, 2026

Target Completion: Approximately 2.5 months (about 10–11 weeks) from notice to proceed

Prepared for competitive proposal / planning purposes

1. RFP Overview

The issuing team seeks proposals from a qualified game-development vendor or development team to design, implement, integrate, test, document, and hand off the complete “Escape the 90s” Unity/C# game.

The project is a 2D side-scrolling game for PC with six independently testable feature areas: UI/HUD and game state; procedural level and obstacle generation; power-ups and abilities; enemies/hazards and combat; meta progression and persistent upgrades; and player control and side-scrolling.

The desired implementation should preserve the project’s documented public interfaces, feature ownership boundaries, event-driven integration approach, Unity/C# platform, and acceptance-oriented testing practices.

1.1 Project at a Glance

Item	RFP Requirement
Project	Escape the 90s
Product type	2D side-scrolling PC game
Engine / language	Unity / C#
Core gameplay	Side-scrolling run, procedural chunks, hazards/enemies, power-ups, player health/death
Meta systems	Run currency, persistent save/load, permanent upgrades
Feature areas	6 major feature owners / subsystems
Current completion window	Approximately 2.5 months / 10–11 weeks
Budget status	Planning estimate; vendor to provide proposed fixed or time-and-materials pricing
Primary deliverable	Integrated, tested, documented PC build and source project

1.2 Procurement Intent

The RFP is intended to obtain a complete implementation rather than isolated code samples. A successful proposal should show how the respondent will deliver an integrated build while keeping feature boundaries clear, interfaces testable, and cross-feature dependencies controlled.

2. Project Background

The project plan defines six roughly equal feature areas at approximately 52 hours each and assigns each feature to a dedicated owner. The architecture is intentionally low-coupling: cross-feature communication uses documented methods, read-only getters, or events instead of shared internal fields or direct access to private implementation details.

The documented architecture identifies the UI/HUD and Game State system as the game shell; the Level Generation system as the source of endless terrain and difficulty; Power-Ups as effect providers; Enemies/Hazards as threats; Meta Progression as the persistent economy and upgrade layer; and Player Controller as the physical center of gameplay.

2.1 Project Objectives

- Deliver a playable and integrated Unity PC game implementing all six project feature areas.
- Provide a coherent endless side-scrolling gameplay loop with menu, pause, game-over, restart, procedural generation, combat, power-ups, and player movement.
- Provide persistent meta progression with currency, at least three upgrade types, save/load, and starting-bonus application.
- Maintain the documented architecture and public interfaces so that features can be independently tested and integrated.
- Demonstrate acceptance through unit, boundary, stress, integration, and end-to-end testing.
- Provide project documentation sufficient for maintenance, evaluation, and future feature additions.

3. Scope of Work

The selected respondent shall provide all work required to complete the following six feature areas and their integration. The descriptions below summarize the supplied project plan and Champion Documents; proposals should identify any requested deviations explicitly rather than silently changing the scope.

3.1 UI/HUD, Game State & Menu System (TL1)

- Own the game shell and top-level state machine: Menu, Playing, Paused, and Game Over.
- Provide main menu, pause/resume, restart, game-over presentation, and HUD behavior.
- Display health, score, and currency; consume typed gameplay notifications without owning the underlying gameplay rules.
- Expose the documented GameStateManager and HUDController boundary, including CurrentState, OnStateChanged, RequestPause, RequestResume, RequestRestart, UpdateScore, and UpdateHealth.
- Use the State pattern with virtual GameState.Enter() and GameState.Update() behavior as documented.

3.2 Procedural Level & Obstacle Generation (TL2)

- Generate and recycle endless level chunks ahead of and behind the player.
- Support chunk pooling and a chunk library with GroundChunk, GapChunk, and RampChunk behavior.
- Scale difficulty with distance and expose read-only distance/difficulty information to dependent systems.
- Avoid visible seams or repeats and support sustained high-speed play without obvious performance degradation.

3.3 Power-Up & Ability System (TL3)

- Spawn pickups and detect collection.
- Provide at least three visually distinct effects, including speed boost, shield, and magnet-style behavior as specified.
- Manage effect duration, stacking, application, and removal through documented player-facing methods.
- Use the documented Strategy/Memento design approach where appropriate to preserve effect selection and temporary-state restoration behavior.

3.4 Enemy/Hazard & Combat System (TL4)

- Populate generated chunks with threats using Ground, Flying, and Turret archetypes.
- Scale spawning from the documented level difficulty signal and validate spawn positions against terrain and minimum player distance.
- Implement patrol, pursuit, attack, search/line-of-sight behavior, attack cooldowns, and collision damage.
- Use EnemyBase dynamic binding for archetype-specific movement/attack behavior and a spawner-based construction approach.
- Publish OnEnemyDefeated events and remove defeated enemies within the documented window.

3.5 Meta Progression & Persistent Upgrades (TL5)

- Track currency earned during runs and persistent currency between sessions.
- Support at least three permanent upgrade types: health, speed, and magnet range.
- Validate purchases by available currency and maximum level; reject invalid purchases without mutating persistent values.
- Save and load progress through a dedicated persistence service with validation, safe defaults, and missing/corrupt-file handling.
- Apply starting bonuses through documented Player Controller methods at run start.

3.6 Player Controller & Side-Scrolling (TL6)

- Provide player input handling, movement physics, camera scrolling, terrain collision, health, and death.

- Support Running, Jumping, and Dashing movement states with dynamic binding.
- Expose PlayerHealth.TakeDamage/Heal, PlayerController.SetSpeedMultiplier, GetPosition, and OnPlayerDied.
- Trigger game-over behavior on death and maintain smooth scrolling without stutter.

4. System Integration & Architecture Requirements

The respondent shall implement the project as one integrated Unity application while preserving the feature boundaries established by the project plan. The following integration rules are mandatory unless an approved change is documented.

Area	Required Integration Behavior
Game shell	TL1 owns top-level game states and the main integration scene.
Level generation	TL2 reads TL6 player position; TL4 and other dependent systems consume documented difficulty/distance outputs.
Power-ups	TL3 affects TL6 only through documented Player Controller methods; TL1 consumes result events for presentation.
Combat	TL4 calls TL6 damage methods and publishes defeat events; it does not modify player health directly.
Persistence	TL5 is the only feature that writes the local save file; SaveDataManager owns serialization, validation, and versioning.
Starting bonuses	TL5 applies unlocked upgrades to TL6 using documented setters at the beginning of a run.
UI	TL1 consumes public events/outputs for score, currency, power-up, and death display/state behavior.
Source ownership	Implementation should remain organized so each major feature can be tested in isolation and integration code does not edit another feature's private internals.

4.1 Public Interface Expectations

The following interfaces are representative of the supplied project specification and shall be treated as typed, documented boundaries. The selected vendor may implement equivalent internal structures only where observable behavior and integration contracts remain unchanged.

Feature	Required Public Boundary
TL1	GameStateManager.CurrentState; OnStateChanged; RequestPause/Resume/Restart; HUDController.UpdateScore/UpdateHealth
TL2	SetDifficulty(float); GetCurrentDistance(); OnChunkSpawned
TL3	SpawnPowerUp(Vector3, PowerUpType); OnPowerUpCollected
TL4	SetSpawnRate(float); OnEnemyDefeated; EnemyBase.TakeDamage/Move behavior
TL5	AddCurrency(int); GetUpgradeLevel(UpgradeType); PurchaseUpgrade(UpgradeType); ApplyStartingBonuses(GameObject); Save/Load
TL6	PlayerHealth.TakeDamage/Heal; PlayerController.SetSpeedMultiplier/GetPosition; OnPlayerDied

5. Technical & Nonfunctional Requirements

- Platform: Unity/C# PC build, with the project kept buildable from the supplied source tree.
- Performance: procedural generation, pooling, combat, and scrolling should remain stable under the documented stress cases.
- Maintainability: use cohesive classes, documented interfaces, and low-coupling feature boundaries.
- Extensibility: new chunk types, enemy types, upgrade types, and movement/effect variants should be addable without rewriting unrelated systems where the documented Protected Variations approach applies.

- Persistence integrity: treat local save data as untrusted input and validate non-negative currency, valid enum values, legal upgrade ranges, and schema/version information.
- Input robustness: game-state transitions and gameplay state changes must reject or safely handle invalid/repeated requests.
- Testing: every externally visible interface shall have observable pass/fail behavior with unit, boundary, stress, or integration tests as applicable.
- Documentation: shipped diagrams, interface contracts, requirements, and implementation notes shall be kept consistent with the final code.

6. User Experience, Presentation & Game Content

The project is a retro-themed 2D side-scroller. The implementation should support a cohesive in-game presentation, but this RFP treats visual polish as part of the final integration and does not replace the gameplay requirements with art-only deliverables.

- Provide readable HUD, menu, pause, game-over, and shop/purchase presentation consistent with the game state.
- Provide visually distinct representations for required power-up types and enemy/hazard archetypes.
- Support the documented gameplay feedback for damage, enemy defeat, power-up collection, currency changes, and state transitions.
- Ensure art/animation assets imported into Unity are organized so they can be integrated into prefabs and scenes without breaking the documented feature ownership boundaries.

7. Testing & Acceptance Requirements

Acceptance shall be based on observable behavior from the supplied feature specifications and Champion Documents. The respondent shall provide evidence of testing before final handoff.

Test area	Acceptance expectation
Game flow	Legal transitions work; illegal pause/resume requests do not create extra state changes; restart returns to a clean Playing state.
HUD	Health, score, and currency reflect accepted inputs within the documented update window; hidden states retain values without displaying gameplay HUD.
Procedural level	Chunk pooling/reuse works; generation remains continuous; difficulty increases with distance; high-speed generation is stress-tested.
Power-ups	At least three effects apply/remove correctly, including stacking/duration boundary behavior.
Combat	At least two enemy/hazard types actively damage the player; spawn rate scales with difficulty; defeat events fire exactly once.
Persistence	Save/load round-trip preserves currency and upgrade levels; missing/corrupt data uses safe defaults without crashing.
Purchases	Insufficient-currency and max-level purchases fail without mutating persistent values.
Starting bonuses	Unlocked bonuses are calculated correctly and applied only through documented Player Controller methods.
Player controller	Run/jump/dash state transitions, health/death, and camera scrolling function continuously without stutter.
Integration	End-to-end scenario supports: earn currency -> purchase upgrade -> save -> restart/load -> apply bonus -> play.

7.1 Required Test Evidence

- Unit-test results for feature-specific logic.
- Boundary test results for zero/max/invalid values and missing/corrupt persistence data.

- Stress-test results for rapid state switching, repeated purchase/save-load activity, enemy counts, and high-speed generation.
- Integration test evidence covering the documented public interfaces.
- A final acceptance checklist mapping the shipped behavior to the RFP requirements.

8. Deliverables

Deliverable	Minimum expected contents
A. Complete Unity project	Source files, scenes, prefabs, scripts, configuration, and assets required to build/run the game.
B. PC / Mobile build	Playable release candidate and final build suitable for demonstration/testing.
C. Feature implementations	All six feature areas integrated and functioning within the agreed scope.
D. Automated/unit test suite	Tests covering public methods, boundary conditions, dynamic-binding behaviors, and critical persistence/gameplay logic.
E. Integration/acceptance evidence	Test results, issue list, resolution status, and final acceptance checklist.
F. Technical documentation	Architecture summary, interface contracts, diagrams, setup/build instructions, persistence notes, and maintenance guidance.
G. Handoff package	Clean project structure, final configuration, known-issues list, and instructions for running/building the system.

9. Schedule & Milestones

The current expected completion window is approximately 2.5 months. For proposal planning, respondents should assume approximately 10–11 weeks from notice to proceed. The exact calendar dates may be set by the issuing team after selection.

Milestone	Target window	Expected outcome
1. Initiation / interface freeze	Week 1	Confirm scope, public interfaces, repository/build setup, schedule, and integration responsibilities.
2. Core systems	Weeks 2–3	Game-state shell, player controller, procedural generation skeleton, core combat/power-up scaffolding.
3. Feature completion	Weeks 4–6	Functional enemies/hazards, power-ups, procedural scaling, UI/HUD flows, meta progression and persistence.
4. Integration	Weeks 7–8	Wire all public interfaces, run start/end flows, HUD updates, currency/upgrade integration, and scene integration.
5. System testing	Weeks 9–10	Boundary, stress, regression, and end-to-end testing; performance and defect correction.
6. Final handoff	Week 11 or earlier	Final build, source package, documentation, acceptance evidence, and known-issues handoff.

The schedule is a planning framework derived from the supplied project timeline and the stated 2.5-month completion window. Respondents may propose a different sequencing only if the proposal identifies dependencies and preserves the required completion window.

10. Estimated Project Budget

The budget in this RFP is an estimate for planning and proposal comparison. The project plan assigns approximately 52 - 60 core hours to each of the six feature areas, for an initial project effort of approximately 312 - 360 core hours.

Budget component	Planning estimate	Basis / assumption
Core implementation effort	312 hours	6 feature areas x approximately 52 hours each, from the project plan.
Project contingency / integration allowance	approximately 15%	Planning allowance for integration, rework, defect correction, and cross-feature coordination.
Overall planning effort	approximately 360 hours	Rounded planning figure for the full project including contingency.
Illustrative labor budget	\$25,000–\$40,000	Planning range, not a quoted price; assumes a blended vendor rate of roughly \$70–\$110/hour.
Third-party software / licenses	Excluded unless proposed	Respondent shall identify any required paid tools, packages, assets, hosting, or services separately.

Budget note: the monetary range is an RFP planning estimate because specify a vendor billing rate or approved dollar ceiling is not known. Respondents should provide a detailed price and assumptions so the issuing team can replace this estimate with an actual proposal value.

11. Vendor / Respondent Proposal Requirements

Proposals should be concise enough to evaluate but detailed enough to demonstrate that the respondent can deliver the complete project within the stated window.

- Executive summary and understanding of the project.
- Proposed technical approach for all six feature areas.
- Integration strategy and approach to preserving public interfaces and low coupling.
- Project schedule mapped to the approximately 10–11 week target.
- Staffing plan identifying roles, estimated effort, and availability.
- Detailed fixed-price or time-and-materials cost proposal, including assumptions and exclusions.
- Testing and quality-assurance approach, including acceptance evidence.
- Documentation and handoff approach.
- Identification of risks, dependencies, and proposed mitigations.
- Relevant Unity/C# game-development experience or representative examples.

12. Proposal Evaluation Framework

The issuing team may evaluate proposals using the following categories. The categories are intended to support a transparent comparison and may be adjusted when the final procurement process is established.

Evaluation category	What will be considered
Technical fit	Ability to meet Unity/C#, architecture, integration, persistence, gameplay, and testing requirements.
Schedule feasibility	Credibility of completing the full scope within approximately 10–11 weeks.
Cost transparency	Clarity of pricing, assumptions, contingency, exclusions, and licensing costs.
Delivery approach	Milestones, communication, source-control practices, testing, and handoff discipline.
Relevant experience	Evidence of successful Unity/C# and 2D gameplay development work.
Risk management	Identification and mitigation of integration, performance, persistence, and schedule risks.

13. Key Assumptions & Constraints

- The project remains a Unity/C# PC build for the scope of this RFP.
- The six major feature areas remain in scope unless formally changed.

- Public interfaces are to be frozen early enough to support integration; later changes require documented coordination and may affect cost/schedule.
- TL1 owns the integration scene, while other feature work may be developed in isolated test scenes/prefabs to reduce scene merge conflicts.
- The local save file is treated as untrusted input and must be validated on load.
- The RFP does not assume online services, multiplayer, cloud saves, or external backend infrastructure unless explicitly proposed as an option.
- Any scope addition that introduces substantial new gameplay systems, platforms, or online services should be quoted as a change or option.

14. Known Project Risks

Risk	Impact	Required mitigation direction
Integration interface drift	Rework / schedule delay	Freeze public signatures early; changes require coordinated review.
Endless generation performance	Frame-rate degradation	Profile pooling and spawn behavior before final integration.
Save-data corruption	Loss of progression / difficult recovery	Versioned schema, input validation, corruption tests, safe defaults.
Uneven feature effort	Schedule overrun	Track actual hours against estimates and review variance regularly.
Integration-scene conflicts	Merge delays / broken build	Maintain clear ownership of the integration scene and isolated test scenes.
Artificial design-pattern use	Unnecessary complexity	Preserve patterns only where they support the documented architecture and requirements.

15. RFP Terms & Submission Notes

- This RFP describes an intended scope and planning estimate; it is not itself a purchase order or guarantee of a minimum spend.
- Respondents shall identify all material assumptions, exclusions, dependencies, and proposed substitutions.
- The issuing team may request clarification, demonstrations, or revised pricing before selection.
- The selected respondent shall not materially alter the documented public interfaces or system boundary without written approval from the issuing team.
- All final source code, build instructions, test evidence, and project documentation required for operation and maintenance shall be included in the agreed handoff unless otherwise negotiated.

Appendix A - Core Feature Summary

Feature	Owner	Primary responsibility	Key completion expectation
TL1	UI/HUD, Game State & Menu	Game shell, HUD, menu, pause, game-over, restart	HUD works; legal state transitions work; restart returns to clean Playing state.
TL2	Procedural Level & Obstacle Generation	Endless chunks, pooling, obstacles, difficulty scaling	Level generates indefinitely without visible seams/repeats and scales difficulty.
TL3	Power-Up & Ability System	Pickups, temporary/permanent effects, duration/stacking	At least 3 distinct power-ups function correctly, including stacking behavior.
TL4	Enemy/Hazard & Combat	Threat spawning, movement/attack, collision damage, defeat	At least 2 active enemy/hazard types damage player and scale with difficulty.
TL5	Meta Progression & Persistent Upgrades	Currency, save/load, upgrade shop, starting bonuses	Currency persists; at least 3 upgrades are purchasable and applied at run start.

Feature	Owner	Primary responsibility	Key completion expectation
TL6	Player Controller & Side-Scrolling	Movement, camera, health, death	Run/jump/dash work smoothly; death triggers correctly; camera scrolls continuously.

End of RFP